

Angular + TypeScript + RxJS.

Proyecto: "SpaceHub - Centro de Control de Misiones Espaciales".

Introducción.

Vivimos en una era emocionante donde la exploración espacial ya no es solo cosa de gobiernos. Empresas privadas como SpaceX, Blue Origin y Rocket Lab están democratizando el acceso al espacio. Misiones a Marte, turismo espacial, satélites de internet y bases lunares ya no son ciencia ficción, son proyectos en desarrollo.

¿quién controla todo esto? Los centros de control de misiones.

Imaginen por un momento ser parte del equipo que monitorea el lanzamiento de un cohete, que sigue la trayectoria de una nave espacial o que recibe los primeros datos de un rover en Marte. Detrás de cada misión exitosa hay cientos de desarrolladores, ingenieros y científicos trabajando con sistemas complejos que procesan datos en tiempo real.

SpaceHub nace de esa inspiración. No es solo otro proyecto de programación; es una puerta al fascinante mundo de la exploración espacial, donde aplicaremos tecnologías de vanguardia para construir nuestro propio centro de control.

Objetivo General: Construir una aplicación web moderna utilizando Angular, TypeScript y RxJS, aplicando principios de programación reactiva para simular un centro de control de misiones espaciales.

Vamos a sentar las bases teóricas. La idea es que entiendan el "por qué" antes del "cómo".

Angular: La Arquitectura.

¿Qué es? Un framework frontend para construir aplicaciones SPA (Single Page

Applications). Se basa en el patrón MVC (Model-View-Controller) pero adaptado a componentes.

- **Arquitectura por Módulos:** Piensen en un módulo como una "caja" que agrupa componentes, servicios y directivas relacionadas (ej: ``MisionesModule``, ``AstronautasModule``). Esto permite Lazy Loading (carga perezosa), que significa que la app solo carga lo que necesita, haciendo que sea más rápida.
- **Componentes:** Son los bloques de construcción de la interfaz de usuario (UI). Cada componente tiene:
 1. `.component.ts` (Lógica: datos, eventos).
 2. `.component.html` (Vista: lo que ve el usuario).
 3. `.component.css/scss` (Estilos).
- **Servicios:** Son clases que no tienen una interfaz de usuario. Su propósito es manejar la lógica de negocio, obtener datos (de una API, de una base de datos, etc.) y compartir información entre diferentes componentes. Se inyectan mediante el sistema de Inyección de Dependencias (DI).

TypeScript Avanzado: El Corazón de Angular.

- Decoradores (``@Component``, ``@Injectable``, ``@Input``, ``@Output``): Son funciones que añaden metadatos a clases, propiedades o métodos. Angular los usa para saber que una clase es un componente o un servicio.
- Genéricos (``<T>``): Permiten crear componentes y servicios que pueden trabajar con cualquier tipo de dato, manteniendo la seguridad de tipos. Ej: ``Observable<Mision[]>``.
- Utility Types (``Partial<T>``, ``Pick<T>``, ``Readonly<T>``): Son tipos predefinidos que nos ayudan a transformar otros tipos. Ej: ``Partial<Mision>`` nos permite tener un objeto que solo tenga algunos campos de ``Mision``, ideal para formularios de edición.

RxJS: Programación Reactiva.

- **Observables:** Son la base. Son "flujos de datos" que emiten valores a lo largo del tiempo. Piensen en ellos como un "stream" o una "cañería" por donde fluye la información.

- **Operadores** (Operators): Son funciones puras que nos permiten transformar, filtrar y combinar estos flujos de datos.
- ``map``: Transforma cada valor emitido. ``pipe(map(data => data.map(m => m.nombre))``
- ``filter``: Filtra los valores. ``pipe(filter(mision => mision.estado === 'activa'))``
- ``switchMap``: Cancela una petición anterior y cambia a una nueva. Es excelente para búsquedas en tiempo real.
- **Subjects**: Son un tipo especial de Observable que también es un "multicast" (puede emitir valores a múltiples suscriptores) y permite emitir valores manualmente. Sirven como un "bus de eventos" para comunicar componentes de forma reactiva.

Patrón de Diseño: Inyección de Dependencias (DI)

Es un patrón donde las dependencias de una clase (como un servicio) se proporcionan desde el exterior en lugar de que la clase las cree internamente (``new Servicio()``).

- **Ventajas**: Hace el código más fácil de testear, más desacoplado y más fácil de mantener.

Pipes Personalizados.

Son funciones que transforman datos en el template HTML. Angular ya tiene algunos como ``DatePipe`` o ``UpperCasePipe``. Nosotros podemos crear los nuestros para formatear fechas de lanzamiento o mostrar el estado de una misión con un color específico.

Requisitos del Proyecto SpaceHub.

- **Temática**: Una aplicación estilo NASA para monitorear misiones espaciales, cohetes y astronautas en tiempo real.
- **Tecnologías**: Angular 18 (o superior), TypeScript 5+, RxJS 7+, Angular Material, Chart.js, y para la simulación de tiempo real, usaremos un ``Subject`` (simula Socket.io).
- **Requerimientos Funcionales (Ids)**:

ID	Requerimiento	Descripción Detallada
S1	Dashboard de Misiones	Vista principal con tarjetas que muestren el nombre, estado (Activa, Completada, Fallida), y un indicador visual del progreso de cada misión.
S2	Mapa de Trayectoria	Una representación visual (usando un `canvas` o un simple `div` con estilos) que simule la órbita de un satélite o la trayectoria de un cohete.
S3	Detalle de Astronautas	Un listado o tarjetas que muestren la información de los astronautas en misión (nombre, rol, estado de salud, etc.) y que se actualice en tiempo real.
S4	Chat de Control	Un componente que simule la comunicación con la misión. Un `Subject` emitirá mensajes aleatorios que se mostrarán en el chat. Los usuarios pueden enviar comandos (ej: "status", "help").
S5	Historial de Lanzamientos	Una tabla con el historial de lanzamientos. Debe tener un campo de búsqueda y filtros por año o estado. Los datos provendrán de un servicio que los genera.
S6	Alertas en Tiempo Real	Un sistema de notificaciones que reaccione a eventos críticos (ej: un cambio de estado a "Crítico" en una misión). Se debe usar un `Subject` de alertas.
S7	CRUD de Misiones	Un formulario que permita Crear, Leer, Actualizar y Eliminar misiones. Los datos se almacenan localmente en un array en el servicio (simulando una base de datos).

Objetivos del Proyecto.

- **Objetivo Técnico:** Aplicar los conceptos de Angular, TypeScript y RxJS en un proyecto integrador.
- **Objetivo de Aprendizaje:** Comprender el flujo de datos reactivo, la comunicación entre componentes, y el uso de servicios para la gestión del estado de la aplicación.

- **Objetivo Profesional:** Desarrollar una aplicación con una arquitectura limpia y escalable, utilizando buenas prácticas de programación y control de versiones (Git/GitHub).

Criterios de Evaluación.

1) Evaluación Continua (60% - Durante la clase):

- Participación y Proactividad: Conexión y participación en el Live Share. Hacer preguntas y mostrar interés.
- Cumplimiento de Entregables Parciales: Subir al menos 2 de los requerimientos funcionando (S1 y otro a elección) en un archivo `.zip` o enlace a su repositorio antes de finalizar la clase.
- Código Limpio y Organizado: Uso de nombres de variables descriptivos, tipado de datos (`: string`, `: number`, `: Mision[]`), y estructura de carpetas lógica.

2) Evaluación Final (40% - Para entregar hasta el martes siguiente):

- Funcionalidad: El proyecto debe cumplir con al menos 5 de los 7 requerimientos.
- Uso de RxJS: Demostrar el uso correcto de Observables, Subjects y operadores (`map`, `filter`, `switchMap`).
- UI/UX: Uso de Angular Material para una interfaz moderna y responsive.
- Informe Técnico: Un documento donde expliquen la arquitectura de su proyecto, las decisiones técnicas tomadas y cómo resolvieron los desafíos.
- Despliegue: El proyecto debe estar desplegado en Vercel o Netlify para ser accesible desde internet.

Actividad/Proyecto: "SpaceHub".

Desarrollar la aplicación "SpaceHub - Donde los sueños espaciales cobran vida".

Pasos para la Realización.

1. Configuración Inicial:

- `ng new space-hub --standalone=false --style=scss`
- `cd space-hub`
- Instalar Angular Material: `ng add @angular/material`
- Instalar Chart.js: `npm install chart.js`

2. Compartir Live Share: Compartir el enlace de Visual Studio Code Live Share con el docente antes de comenzar a programar para que pueda observar el proceso (ver "Entregables").

3. Estructura de Carpetas (Sugerida):

```
src/
├── app/
│   ├── core/           // Servicios, guards, modelos
│   │   ├── models/     // Interfaces: mission.interface.ts, astronaut.interface.ts
│   │   └── services/    // mission.service.ts, astronaut.service.ts, alert.service.ts
│   ├── shared/         // Componentes reutilizables
│   │   ├── components/ // (Header, Footer, AlertComponent)
│   │   └── pipes/       // (StatusColorPipe, DateFormatPipe)
│   ├── features/       // Módulos con Lazy Loading
│   │   ├── dashboard/  // (S1)
│   │   ├── missions/   // (S5, S7)
│   │   └── astronauts/ // (S3)
│   └── app-routing.module.ts // Configuración de rutas
```

Ejemplo Práctico: El Servicio de Alertas.

Para que vean cómo se hace, vamos a construir el sistema de alertas paso a paso.

Paso 1: Modelo (Interface)

```
// core/models/alert.interface.ts
export interface Alert {
  id: number;
```

```
message: string;
type: 'info' | 'warning' | 'danger';
timestamp: Date;
}
```

Paso 2: Servicio (Subject)

```
// core/services/alert.service.ts
import { Injectable } from '@angular/core';
import { Subject } from 'rxjs';
import { Alert } from '../models/alert.interface';

@Injectable({
  providedIn: 'root'
})
export class AlertService {
  private alertSubject = new Subject<Alert>();

  // Exponemos el observable para que los componentes se suscriban
  alerts$ = this.alertSubject.asObservable();

  // Método para emitir una nueva alerta
  sendAlert(message: string, type: Alert['type'] = 'info') {
    const newAlert: Alert = {
      id: Date.now(),
      message,
      type,
      timestamp: new Date()
    };
    this.alertSubject.next(newAlert);
  }

  // Método para simular alertas aleatorias
```

```
startRandomAlerts() {  
  const messages = [  
    '¡Atención! Temperatura en la cápsula crítica.',  
    'Misión Apolo 12 ha entrado en órbita.',  
    'Pérdida de señal con el satélite OVNI-3.',  
    'Comunicación establecida con la Estación Espacial.'  
  ];  
  setInterval(() => {  
    const randomMessage = messages[Math.floor(Math.random() * messages.length)];  
    const types: Alert['type'][] = ['info', 'warning', 'danger'];  
    const randomType = types[Math.floor(Math.random() * types.length)];  
    this.sendAlert(randomMessage, randomType);  
  }, 5000); // Cada 5 segundos  
}  
}
```

Paso 3: Componente (Suscripción)

```
// shared/components/alert-list/alert-list.component.ts  
import { Component, OnInit } from '@angular/core';  
import { AlertService } from '../../../core/services/alert.service';  
import { Alert } from '../../../core/models/alert.interface';  
  
@Component({  
  selector: 'app-alert-list',  
  templateUrl: './alert-list.component.html',  
  styleUrls: ['./alert-list.component.scss']  
})  
export class AlertListComponent implements OnInit {  
  alerts: Alert[] = [];  
  
  constructor(private alertService: AlertService) {}  
}
```



```
ngOnInit() {  
  this.alertService.alerts$.subscribe(newAlert => {  
    this.alerts.unshift(newAlert); // Agregamos al principio  
    // Opcional: eliminar la alerta después de 10 segundos  
    setTimeout(() => {  
      this.alerts = this.alerts.filter(a => a.id !== newAlert.id);  
    }, 10000);  
  });  
}
```

Entregables.

1. Enlace de Live Share: Compartir el enlace en el comentario privado de la actividad específica en la plataforma educativa antes de empezar a codificar.
2. Avance Parcial (Durante la Clase):
 - Subir un archivo `.zip` del proyecto (sin la carpeta `node\_modules`) a la plataforma o a un enlace de Google Drive.
 - O bien, compartir el enlace al repositorio de GitHub donde estén subiendo el avance.
3. Entrega Final (Hasta el martes siguiente):
 - Repositorio GitHub: Repositorio público con el código completo, un `README.md` bien documentado y el historial de commits.
 - Despliegue: URL de la aplicación en Vercel (o Netlify).
 - Informe Técnico: Documento en formato PDF o Markdown que contenga:
 - Introducción y objetivos.
 - Explicación de la arquitectura utilizada.
 - Dificultades encontradas y cómo las resolvieron.
 - Capturas de pantalla de la aplicación.
 - Conclusiones y aprendizajes.